FITMATE · DEVELOPER GUIDE

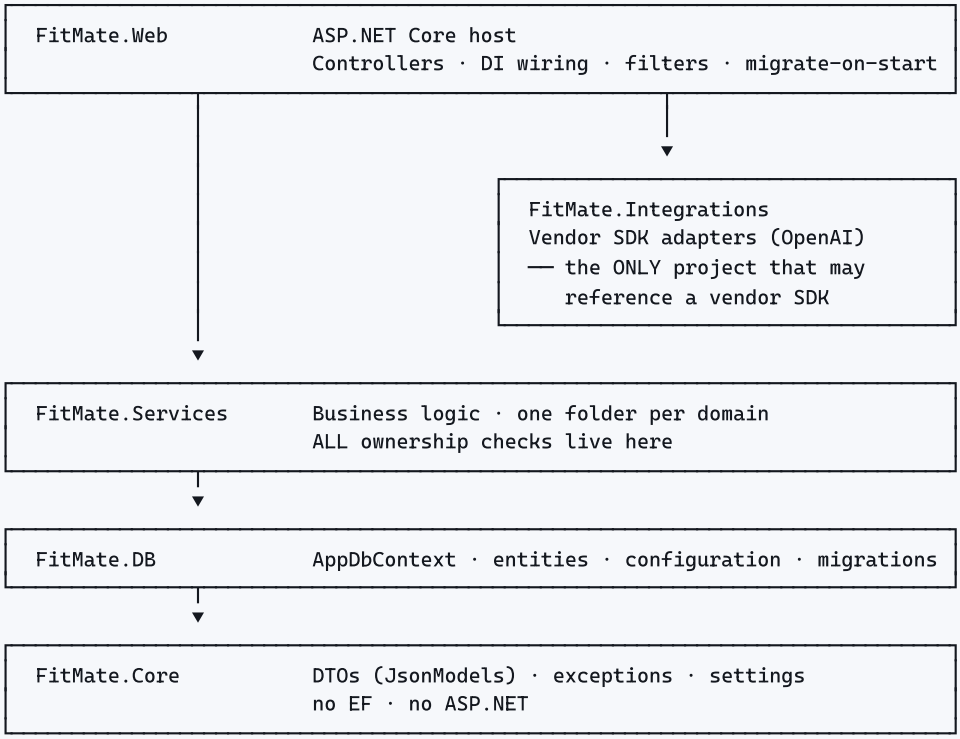
Flow & Call-Path Guide

How a request travels through FitMate — where every piece lives, what calls what, and which guard rails fire along the way. Covers the AI coach, subscriptions and entitlements, program plans, and the hosting changes that support them.

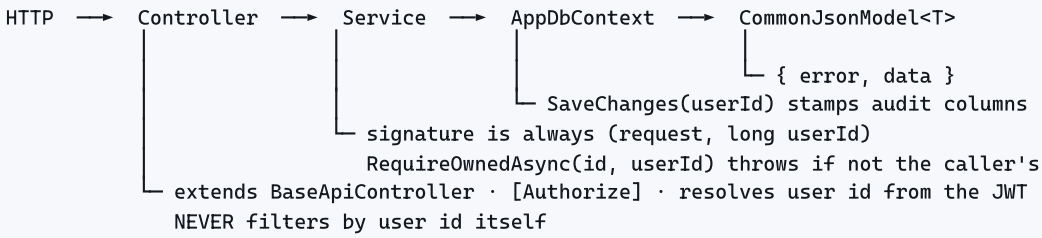
BRANCH	`feat/ai-coach-subscriptions`
SCOPE	AI coach · subscriptions & entitlements · program plans · training profiles · admin
COMPANION	`docs/architecture/*.md`
VERIFIED AGAINST	source at time of writing — not the implementation plans

01 The map

Five server projects. Dependencies point strictly downward — nothing lower ever references anything above it. If you know this diagram you can place any file in the codebase.



The shape of every request



> **THE ONE RULE THAT MATTERS MOST**
>
> Ownership is enforced in the **service**, never the controller. Every service has a `RequireOwnedAsync(id, userId)` helper that throws if the row is not the caller's. A controller that filters by user id is a bug waiting to be copy-pasted into one that forgets.

Where exceptions become status codes

One place: `FitMate.Web/Attributes/LogApiErrorAttribute.cs`, registered globally as an exception filter.

EXCEPTION	STATUS	WRITTEN TO THE ERRORS TABLE?
`SubscriptionLimitExceededException`	**429**	No — expected outcome, logged at Warning
`SubscriptionFeatureDisabledException`	**403**	No — expected outcome, logged at Warning
`FitMateException`	**400**	No — the user caused it and can fix it
`anything else`	**500**	**Yes**, with the full stack trace

Model-validation 400s never reach the filter — they short-circuit in `InvalidModelStateResponseFactory`, which logs at Warning under `FitMate.Web.ModelValidation` so they still reach the error grid.

02 AI coach — sending a message

The single most important flow in the branch. One method is its spine:
`FitMate.Services/AI/AIOrchestrator.cs → SendAsync`

`POST` `/api/ai/conversations/{id}/messages` → `AIController.SendMessage` → `AIOrchestrator.SendAsync`

THE BOUNDED TOOL LOOP	
1	**Plan gate** `entitlementService.RequireFeatureAsync(userId, AIChat)` 403 if the plan does not include AI chat.
2	**Quota gate — reserve, don't spend** `usageService.ReserveAsync(userId, AIChat, 1)` 429 if the monthly allowance is exhausted. Both gates run *before* any provider call, so a user who cannot use the feature never costs money.
3	**Persist the user's message** `conversationService.AddUserMessageAsync(conversationId, content, userId)` Conversation ownership is checked here — before a run row exists, so a rejected request leaves no audit noise.
4	**Open the audit row** `runService.StartAsync(userId, conversationId, provider, model, promptVersion)` Everything from here is attributable to this AIRun.
5	**Build the context** `contextBuilder.BuildAsync(conversationId, userId)` System prompt + the last 30 user/assistant messages. Tool traffic is persisted but never replayed from history.
6	**Resolve the allow-list** `toolRegistry.GetDefinitions(toolContext)` Only the tools available to this user, filtered by each handler's `IsAvailable`.
7	**↻ Loop — at most 6 iterations, whole loop under a 90s CTS** `completionProvider.CompleteAsync(request, timeout.Token)` The only network call. Behind `IAICompletionProvider`, so tests swap in a fake.
8	**Meter every iteration** `runService.AddUsageAsync(run.Id, response.Usage, response.ProviderRequestId)` Tokens accumulate per iteration, not once at the end — a run that later fails still reports what it burned.
9	**EXIT — no tool calls: the model answered** `AddAssistantMessageAsync → runService.CompleteAsync → usageService.CommitAsync(reservation.Id)` Returns the message, the tool names used, any pending action cards, and a fresh usage summary.
10	**EXIT — too many tool calls (> 12 in one run)** `runService.MarkLimitExceededAsync(run.Id, "tool_call_limit") → usageService.ReleaseAsync(...)` Throws `AIToolLimitExceededException`.
11	**Execute each tool call** `toolRegistry.ExecuteAsync(toolCall, toolContext, timeout.Token)` Unknown tool → recorded as Rejected and the failure is handed back to the model, which can apologise or try another route. A throwing tool does not kill the run.
12	**Persist call + result, then feed them back** `AddToolCallMessageAsync / AddToolResultMessageAsync → messages.Add(...)` Written to `AIMessages` for audit, appended in memory so the model sees the result next iteration.

13

EXIT — fell out of the loop

`MarkLimitExceededAsync(run.Id, "tool_iteration_limit") → ReleaseAsync(...)`

The model never stopped calling tools within 6 iterations.

WHY THE RESERVATION IS THE DESIGN

There are exactly three exits (9, 10, 13) plus a catch-all `catch`. Every one of them finalises the reservation exactly once — commit on success, release on every failure. That is what stops a provider outage silently eating a user's monthly quota, and why `CommitAsync`/`ReleaseAsync` are idempotent.

Limits, and where they are enforced

AIOPTIONS SETTING	DEFAULT	ENFORCED BY
`TimeoutSeconds`	90	CancellationTokenSource wrapping the whole loop
`MaximumToolIterations`	6	the `for` bound
`MaximumToolCallsPerRun`	12	running total across iterations
`MaximumConversationMessages`	30	history window in `AIContextBuilder`
`StoreRawProviderPayload`	false	whether raw provider JSON is kept on the run

03 Tools — what the model may touch

`FitMate.Services/AI/Tools/AIToolRegistry.cs` is the allow-list. Registration is explicit in `AIToolServiceCollectionExtensions`, never assembly-scanned — so adding a handler class cannot widen what the AI can do by accident. It must also be named there.

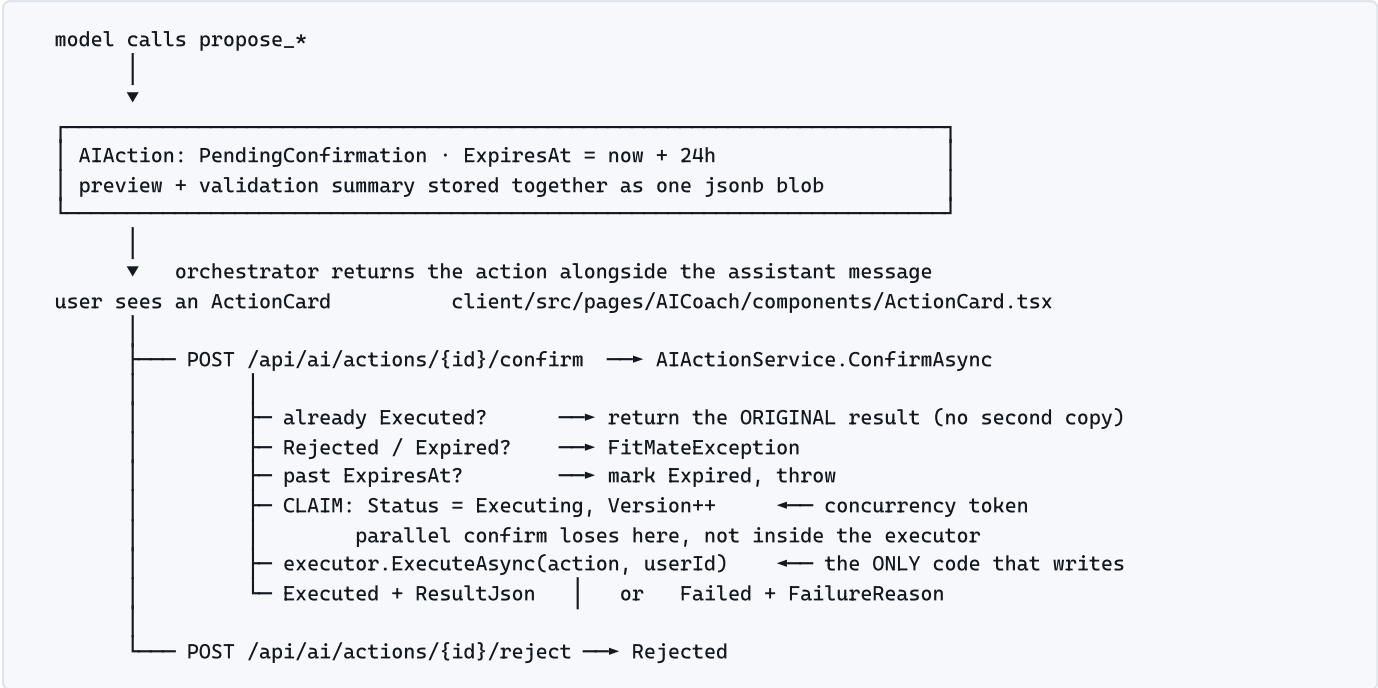
TOOL	HANDLER	BEHAVIOUR
`get_training_profile`	`ReadOnly/`	Execute immediately. Ownership from `AIToolContext.UserId`. No confirmation.
`get_active_program`	`ReadOnly/`	
`get_program_calendar`	`ReadOnly/`	
`get_subscription_usage`	`ReadOnly/`	
`search_exercises`	`ReadOnly/`	
`get_recent_workouts`	`ReadOnly/`	
`get_exercise_history`	`ReadOnly/`	
`get_workout_templates`	`ReadOnly/`	
`propose_exercise`	`Proposals/`	**Write nothing.** Validate → look for duplicates → create a pending `AIAction` → return `requiresConfirmation: true`.
`propose_workout`	`Proposals/`	
`propose_workout_template`	`Proposals/`	
`propose_program_plan`	`Proposals/`	
`propose_program_update`	`Proposals/`	
`report_unsupported_request`	`Unsupported/`	Writes only to the admin backlog, so it needs no confirmation.

> **PATTERN WORTH COPYING**
>
> `ProposeExerciseToolHandler` sets `payload.IsGlobal = false` before validating. The model **cannot** create a global exercise no matter what it sends — only the admin-only endpoint does that. Scope decisions are never left to model output.

Every attempt — allowed, rejected or failed — writes an `AIToolExecution` row with arguments, result, status and duration, redacted first by `AI/AIRedactionService.cs`. Redaction protects the *audit trail*; it runs on the way into the database, not on the way out to the provider.

04 Proposals — the confirmation flow

The model proposes; the user disposes. This is the only path by which AI-originated data is written, and the executors are the only code that writes it.



> **WHY CLAIM-THEN-EXECUTE**
>
> Two taps on *Confirm* both load the row, but only one wins the `Version` check. The loser reloads and either returns the winner's result or throws `AIActionAlreadyExecutedException`. Without the claim, a double-tap creates two program plans.

Executor per action type

AIACTIONTYPE	EXECUTOR	WRITES
`CreatePersonalExercise` `CreateGlobalExercise`	`CreateExerciseActionExecutors`	Exercises (+ aliases)
`CreateWorkout` `CreateWorkoutTemplate`	`CreateWorkoutActionExecutors`	Workouts / WorkoutTemplates
`CreateProgramPlan`	`CreateProgramPlanActionExecutor`	ProgramPlans + rules + days
`UpdateProgramPlan`	`UpdateProgramPlanActionExecutor`	reshapes an active plan
`GenerateExerciseImage`	*none*	declared but never implemented

Expiry is **lazy** — `ExpireIfDueAsync` runs on read and on confirm. There is no background job, so an action expires the next time someone looks at it, not on a timer.

05 Entitlements & usage

Two questions, two services. Anything metered asks both, in this order. `FitMate.Services/Subscriptions/`

"Is this user allowed to?" — EntitlementService

```
ResolvePlanAsync(userId)          first match wins
──────────────────────────────────────────────────────────────────────────
 1. active UserPlanOverride   (IsActive, StartsAt ≤ now, EndsAt null|future)  → AdminOverride
 2. active UserSubscription   (Active | Trialing)                             → Subscription
 3. the seeded Free plan                                                       → FreePlan
──────────────────────────────────────────────────────────────────────────
 safety net: plan missing or IsActive == false  ──►  fall back to Free
             a deactivated plan must never grant more than Free
──────────────────────────────────────────────────────────────────────────
 result cached in IMemoryCache for 60s per user  ── call Invalidate(userId) after a grant
```

TODAY'S REALITY

Stripe was never built, so nothing writes `UserSubscription`. In practice every user resolves to **Free** unless an admin grants an override.

"Have they got any left?" — UsageService

```
ReserveAsync(userId, feature, qty)
   ├─ entitlement disabled?            ──► SubscriptionFeatureDisabledException   403
   ├─ qty > MaximumPerRequest?          ──► FitMateException                        400
   ├─ ExpireStaleReservationsAsync      reclaim anything abandoned > 15 min ago
   ├─ Used + Reserved + qty > limit?    ──► SubscriptionLimitExceededException    429
   └─ bucket.Reserved += qty; Version++      ──►  UsageReservation (Active)
                                               +  UsageEntry (Reservation)

CommitAsync(id)    Reserved −= qty; Used += qty    + UsageEntry (Commit)     idempotent
ReleaseAsync(id)   Reserved −= qty                 + UsageEntry (Release)    idempotent
```

THE CONCURRENCY TOKEN DOES THE REAL WORK

`UsageBucket.Version` is a concurrency token, so two simultaneous reservations serialise. The loser catches `DbUpdateConcurrencyException`, clears the change tracker and **re-evaluates the limit against fresh numbers** — it does not simply retry the write. That is what stops both requests spending the last unit. Up to 3 attempts, then a retryable error.

Two consumption shapes

SHAPE	USED BY	HOW
Metered `MonthlyLimit`	`AIOrchestrator`	Reserve up front → commit on success → release on any failure. Tracked in a `UsageBucket` per user, per feature, per month.
Standing ceiling `HardLimit`	`ProgramPlanService` `.RequireActivationEntitlementsAsync`	No bucket, no reservation — count the rows that exist and compare against the limit. The current row count *is* the usage.

Error contract

SITUATION	STATUS	PAYLOAD IN DATA
Feature not on the plan	**403**	`subscription_feature_disabled` — `feature`, `upgradeAvailable`
Monthly quota exhausted	**429**	`subscription_limit_reached` — `feature`, `limit`, `used`, `reserved`, `resetsAt`

Both carry real numbers so the client can render an upgrade prompt instead of a generic message.

06 Program plans

The module answers one question on the home screen: **what am I training today?** The answer is always read from persisted `ProgramPlanDay` rows — never computed on the fly, never asked of the AI.

Activation: where the calendar is born

```
POST /api/program-plans/{id}/activate  ──►  ProgramPlanService.ActivateAsync
      │
      ├─ RequireActivationEntitlementsAsync
      │      ├─ ActiveProgramPlans        how many plans may be active at once
      │      └─ ProgramPlanDurationMonths fixed-length plans only
      │
      └─ scheduleService.GenerateDays(plan, from, toInclusive)   ── pure, no DB, no clock
            ├─ EndDate set   ──► generate ALL days
            └─ EndDate null  ──► generate a rolling 28-day horizon
                                 (OpenEndedHorizonDays), topped up later by
                                 EnsureUpcomingDaysAsync at the request boundary
```

Generation rules — ProgramPlanScheduleService

```
FixedWeekdays   rule.DayOfWeek == date.DayOfWeek
                && (daysSinceStart / 7) % max(1, rule.WeekInterval) == 0     every N weeks

Rotation        rule.RotationDayIndex == (daysSinceStart % cycleLength) + 1
                cycleLength = max RotationDayIndex across the rules
                an A/B/C/rest rotation is a 4-slot cycle that ignores weekdays entirely

CustomCalendar  returns []  - days are supplied explicitly, not projected

Rest rules are DROPPED, not materialised: a rest day is the absence of a row.
```

Reading today

```
GET /api/program-plans/active/today?date=2026-08-05      ◄── client sends its LOCAL date
      │                                                        server falls back to UtcNow
      ├─ dayService.MarkMissedDaysAsync(userId, date)
      │     marks days STRICTLY BEFORE the reference date still Scheduled|Rescheduled
      │     OptionalWorkout ──► Skipped, not Missed   (an optional day you skipped is not a failure)
      │
      ├─ EnsureUpcomingDaysAsync(plan, date)     top up the open-ended horizon
      └─ return the persisted row for that date
```

> **NO BACKGROUND JOBS WERE BUILT**
>
> Missed-day marking, horizon top-up, reservation expiry and AI-action expiry all run **lazily at the request boundary**. A user who never opens the app never gets their days marked missed, and their calendar stops being extended.

Lifecycle

```
Draft ──activate──► Active ──pause──► Paused ──activate──► Active
                      ├──complete──► Completed
                      └──cancel────► Cancelled
Draft ──delete──► gone            (only drafts can be deleted)
```

> **RESHAPING NEVER REWRITES HISTORY**
>
> `UpdateActiveScheduleAsync` regenerates days from `effectiveFrom` — but only those still in `Scheduled` status. Completed, Started, Missed, Skipped and Rescheduled days survive untouched. The AI's `propose_program_update` path goes through the same method, so the coach cannot erase training history either.

Day actions

ENDPOINT	RULE
`POST` `/{id}/start`	Plan must be Active. Rest days and days with no template are rejected. Creates the `Workout` and returns its id.
`POST` `/{id}/move`	Target must be inside the plan range and free. Day becomes `Rescheduled`.
`POST` `/{id}/skip`	Deliberate, user-initiated.
`POST` `/{id}/restore`	Only from Skipped or Missed. Returns to `Scheduled` if still future, else `Rescheduled`.

07 Where does what live

A lookup table for “I need to change X — which files?”

Server

IF YOU'RE CHANGING…	START HERE
What the AI is allowed to do	`Services/AI/Tools/AIToolServiceCollectionExtensions.cs` register the handler — a class alone is not enough
The AI loop, limits, gating	`Services/AI/AIOrchestrator.cs` · `Core/Settings AIOptions`
The system prompt	`Services/AI/AIPromptBuilder.cs` (bump SystemPromptVersion)
What gets written after a confirm	`Services/AIActions/Executors/`
Swapping the model vendor	`Integrations/AI/OpenAI/` + one DI line. Nothing else.
Plan limits / new feature flag	`DB/Enums/SubscriptionFeature.cs` · `Web/SeedData/plans.json`
How quota is counted	`Services/Subscriptions/UsageService.cs`
Who gets which plan	`Services/Subscriptions/EntitlementService.cs`
Calendar generation maths	`Services/ProgramPlans/Schedules/ProgramPlanScheduleService.cs`
Plan lifecycle / entitlement gates	`Services/ProgramPlans/Plans/ProgramPlanService.cs`
Day transitions, missed marking	`Services/ProgramPlans/Days/ProgramPlanDayService.cs`
HTTP status for an exception	`Web/Attributes/LogApiErrorAttribute.cs`
What reaches the error grid	`Web/Infrastructure/SerilogDatabaseSink.cs` · `Program.cs overrides`
A new DTO for the client	`Core/JsonModels/<Feature>/` → build → `npm run process-types`
DI registration	`Web/Program.cs` (~line 300, grouped by domain)

Client

PAGE	PATH UNDER CLIENT/SRC/PAGES/
AI coach	`AICoach/` — `ActionCard`, `MessageBubble`, `ToolActivityIndicator`, `useAICoachPage`
Program list & detail	`Program/`
Program builder	`ProgramBuilder/` — one editor per schedule type
Program calendar	`ProgramCalendar/`
Subscription & usage	`Subscription/` — `UsageBar`
Training profile	`Profile/TrainingProfile.tsx`
Admin	`AdminPanel/`

NEVER HAND-WRITE A BACKEND TYPE

Request/response types come from `client/src/types/backend.ts`, generated by Reinforced.Typings during `dotnet build` of FitMate.Web, then split by `npm run process-types`. If a type is missing or stale, build the backend — do not add a local interface in a service file.

08 Hosting — the parts that bite

FitMate runs as a container on Railway behind a TLS-terminating proxy. The container filesystem is ephemeral: anything that must survive a deploy lives in the database or blob storage.

Data protection keys

```
builder.Services.AddDataProtection()
    .PersistKeysToDbContext<AppDbContext>()      ── key ring in Postgres, not the container FS
    .SetApplicationName("FitMate");              ── pins the key-derivation discriminator
```

The default store writes to `/root/.aspnet/DataProtection-Keys`, which is discarded on every deploy. Each release generated a fresh key ring, and anything protected by the previous one stopped validating. In production this silently invalidated **every password-reset link that had already been emailed** — three key rings in the first three weeks.

DEPENDS ON THE KEY RING	DOES NOT
Identity token providers — `GeneratePasswordResetTokenAsync`. The token is not stored anywhere; it *is* a protected payload the server unprotects later.	JWTs, signed with `Jwt:SigningKey` from configuration, stable across deploys. That is why logins were unaffected — and why the bug went unnoticed, since it only hit users who could not log in to complain.

Behind the proxy

```
app.UseForwardedHeaders(...)   ── MUST be first: XForwardedProto | XForwardedFor
app.UseHttpsRedirection();        KnownNetworks/KnownProxies cleared — Railway's proxy IP is not stable
```

Without it the app only ever sees plain HTTP, so `UseHttpsRedirection` tries to redirect every already-secure request, finds no HTTPS port, and logs a warning per request.

Serilog overrides match the full logger category

> **THE TRAP**
>
> An override keyed on a namespace that does not exist matches **nothing**, silently. An earlier filter read `Microsoft.AspNetCore.HttpsRedirection`; the real category is `Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware`. It never fired, and the warning it was meant to suppress accumulated until the production error grid contained nothing but framework noise.
>
> When adding an override, read the category from a real log row — the `Source` column in `Errors` — rather than guessing from the type name.

Not built

PLAN	WHAT THAT MEANS IN THE CODE
09 — Stripe billing	`PlanPrice.StripePriceId` and `UserSubscription` are read by the entitlement resolver but nothing writes them. Paid plans are reachable only via an admin override.
10 — Vision & images	`AIActionType.GenerateExerciseImage` has no executor; no `propose_exercise_image` tool. `IAIImageProvider` is wired but never called.
11 — Background jobs	Everything job-shaped runs at the request boundary instead. See the warning in §06.

Two tools named in the roadmap were never implemented: `get_training_snapshot` and `propose_exercise_image`. There is no `ITrainingSnapshotService` — the model pulls training data through the read-only tools instead.